Grado en Ingeniería Informática

Fundamentos de Programación II 25/26

Hito 3

Proyecto práctico: Hito 3

Resumen

El objetivo de esta práctica es afianzar los conocimientos adquiridos sobre manejo de excepciones y trabajo básico con ficheros, aplicándolos sobre el sistema orientado a objetos desarrollado en los hitos anteriores. El trabajo se plantea por parejas y se centra en seguir ampliando el núcleo de un videojuego RPG (Role-Playing Game).

1. Objetivos

- Afianzar el uso de excepciones propias del sistema para representar situaciones anómalas del dominio.
- Trabajar con excepciones de entrada/salida de Java en operaciones de lectura y escritura de ficheros.
- Adaptar un sistema ya existente manteniendo la coherencia entre análisis, diseño e implementación.
- Consolidar el proceso de trabajo por fases: análisis, diseño e implementación.

2. Normas de trabajo

El ejercicio se realizará por parejas y se entregará en la tarea habilitada en Campus Virtual. Se deberán seguir estas normas:

- **Entrega**
  - Todos los archivos deberán comprimirse en un único archivo ZIP.
  - El nombre del ZIP debe seguir el formato `LX_XX.zip` según el grupo de Campus Virtual.
  - El ZIP deberá incluir:
    - Imagen del diagrama de clases y relaciones UML (`.png` o `.jpg`).
    - Únicamente la carpeta `src` con los archivos `.java`.
- **Restricciones de implementación**
  - No utilizar múltiples `return` en una misma función.
  - No utilizar `continue`.
  - No utilizar `break`, salvo en sentencias `switch`.
  - No utilizar `ArrayList`.
- Queda totalmente **prohibido el uso de Inteligencia Artificial** para realizar estas prácticas. Cualquier indicio de su utilización será penalizado.

3. Fases del desarrollo orientado a objetos

Se pide seguir las fases del desarrollo de software orientado a objetos:

- **Análisis:** En esta fase se identifican los requisitos del sistema. Intentamos encontrar los objetos y relaciones adecuados para el problema propuesto. Investigamos el problema centrándonos en los conceptos del dominio del problema, sin preocuparnos por la implementación. Es importante entender el problema y definir claramente los requisitos antes de pasar a la siguiente fase.
- **Diseño:** En esta fase se define la arquitectura del sistema. Partiendo de los objetos y relaciones identificados en el análisis, se diseña la estructura del sistema. Se definen las clases, sus atributos y métodos (aún sin pensar en la implementación, solo en aquella funcionalidad que cada clase debe tener), así como las relaciones entre ellas (herencia, composición, etc.). Es importante diseñar un sistema modular y flexible que permita cambios futuros sin afectar a toda la aplicación. En esta fase es fundamental el uso de diagramas UML (especialmente el diagrama de clases).
- **Implementación:** Únicamente cuando tenemos claro el diseño, pasamos a la implementación. En esta fase se escribe el código fuente de la aplicación siguiendo el diseño establecido. Es importante seguir buenas prácticas de programación y mantener el código limpio y organizado.

4. Desarrollo de la práctica

En este hito se parte del sistema desarrollado en el hito anterior y se incorpora una nueva capa de trabajo centrada en el manejo de errores mediante excepciones. La funcionalidad general del videojuego se mantiene, pero ahora determinadas situaciones anómalas deberán representarse y gestionarse de forma explícita. Además, se añadirá una nueva opción que permitirá exportar a un fichero de texto el estado actual del personaje con el que se esté trabajando, siguiendo el mismo formato general ya utilizado en los hitos anteriores.

En esta fase se trabajará con dos tipos de excepciones. Por un lado, se deberán implementar excepciones propias del sistema para representar errores del dominio del problema. Por otro, se trabajará también con las excepciones de entrada/salida de Java asociadas a la lectura y escritura de ficheros.

Las excepciones propias que deberán incorporarse son las siguientes:

- `InventarioLlenoException`: se producirá cuando se intente añadir un objeto a un inventario que ya no dispone de huecos libres.
- `ObjetoNoEquipableException`: se producirá cuando se intente equipar un objeto que no puede formar parte del equipo del personaje.
- `PosicionVaciaException`: se producirá cuando se intente utilizar una posición del inventario o de la tienda que exista, pero que no contenga ningún objeto.
- `DineroInsuficienteException`: se producirá cuando un personaje intente comprar un

objeto sin disponer de dinero suficiente.

- `TipoObjetoInvalidoException`: se producirá cuando, durante la lectura de los ficheros del sistema, aparezca un tipo de objeto no reconocido.

La nueva funcionalidad de escritura permitirá guardar en un fichero la información completa del personaje actual, incluyendo sus datos, el equipo que tiene equipado y el contenido de su inventario. Ese fichero deberá seguir el mismo formato general que ya se ha utilizado para representar personajes en los hitos anteriores, de forma que mantenga coherencia con el sistema ya desarrollado.

Se pide actualizar el diagrama UML para reflejar los cambios introducidos en este hito, implementar las nuevas excepciones y adaptar las clases necesarias para integrarlas correctamente en el funcionamiento del programa. También se deberá incorporar la exportación del personaje actual a fichero y comprobar que el sistema sigue funcionando correctamente cuando se producen tanto errores propios del dominio como errores relacionados con la entrada/salida.